Tenno: Data Privacy Act

Falcon-7B Phase 2 demonstration Notebook

This notebook demonstrates the Phase 2 Falcon configuration used in the final interactive prototype. Unlike the baseline Falcon model, this version applies the Phase 2 QLoRA adapter and uses Top-K=5 retrieval to improve contextual coverage during live legal question answering.

1. Install Requirements

```
!pip install -q gradio transformers accelerate bitsandbytes peft
sentence-transformers faiss-cpu
```

This installs the UI framework (Gradio), the model inference stack (Transformers, Accelerate, BitsAndBytes, and PEFT for LoRA adapters), and the retrieval stack (SentenceTransformer and FAISS).

2. Mount Google Drive

```
from google.colab import drive
drive.mount("/content/drive", force_remount=True)
```

```
Mounted at /content/drive
```

Google Drive is mounted so the notebook can load your saved FAISS index + metadata and store Hugging Face cache files persistently.

3. Set Project Paths + Persistent HF Cache

```
import os, json

PROJECT_DIR = "/content/drive/MyDrive/Mapua /3rd Year Term 2/CSS181-
3/NLP_Project"
CHUNKS_DIR  = os.path.join(PROJECT_DIR, "Chunks")
MODELS_DIR  = os.path.join(PROJECT_DIR, "Models")
RESULTS_DIR = os.path.join(PROJECT_DIR, "Results")

PHASE2_ADAPTER_DIR = os.path.join(MODELS_DIR,
"falcon_finetune_adapter_v2_topk5")
print(" PHASE2_ADAPTER_DIR exists?",
os.path.exists(PHASE2_ADAPTER_DIR))
```

```
os.makedirs(MODELS_DIR, exist_ok=True)
os.makedirs(RESULTS_DIR, exist_ok=True)

FAISS_INDEX_PATH = os.path.join(CHUNKS_DIR, "faiss_index.bin")
FAISS_META_PATH  = os.path.join(CHUNKS_DIR, "faiss_metadata.json")

HF_CACHE_DIR = os.path.join(MODELS_DIR, "hf_cache")
os.makedirs(HF_CACHE_DIR, exist_ok=True)

os.environ["HF_HOME"] = HF_CACHE_DIR
os.environ["HUGGINGFACE_HUB_CACHE"] = HF_CACHE_DIR
os.environ["TRANSFORMERS_CACHE"] = HF_CACHE_DIR

print("□ PROJECT_DIR:", PROJECT_DIR)
print("□ FAISS_INDEX_PATH exists?", os.path.exists(FAISS_INDEX_PATH))
print("□ FAISS_META_PATH exists?", os.path.exists(FAISS_META_PATH))
print("□ HF cache:", HF_CACHE_DIR)
```

```
 PHASE2_ADAPTER_DIR exists? True
□ PROJECT_DIR: /content/drive/MyDrive/Mapua /3rd Year Term 2/CSS181-3/NLP_Project
□ FAISS_INDEX_PATH exists? True
□ FAISS_META_PATH exists? True
□ HF cache: /content/drive/MyDrive/Mapua /3rd Year Term 2/CSS181-3/NLP_Project/Models/hf_cache
```

The Phase 2 adapter directory points to the fine-tuned Falcon LoRA weights produced during the Top-K=5 experiment and stored in Google Drive for reuse during demo inference.

4. Load FAISS Index + Metadata

```
import faiss

if not os.path.exists(FAISS_INDEX_PATH):
    raise FileNotFoundError(f"Missing FAISS index: {FAISS_INDEX_PATH}")

if not os.path.exists(FAISS_META_PATH):
    raise FileNotFoundError(f"Missing FAISS metadata: {FAISS_META_PATH}")

index = faiss.read_index(FAISS_INDEX_PATH)

with open(FAISS_META_PATH, "r", encoding="utf-8") as f:
    meta = json.load(f)
```

```
print("□ FAISS vectors:", index.ntotal)
print("□ Metadata rows:", len(meta))
```

```
□ FAISS vectors: 198
□ Metadata rows: 198
```

FAISS loads the vector index of your legal chunks. Metadata is loaded in the same order as embeddings so we can map FAISS IDs back to the original chunk text and source document details.

5. Load SentenceTransformer Embedder

```
from sentence_transformers import SentenceTransformer
import numpy as np

EMBED_MODEL_NAME = "all-MiniLM-L6-v2"
embedder = SentenceTransformer(EMBED_MODEL_NAME)

print(f"□ Embedder loaded: {EMBED_MODEL_NAME}")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"94cdfb21603540d8bc291e52eb89579d","version_major":2,"version_minor":0}

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
```

```
 Embedder loaded: all-MiniLM-L6-v2
```

The embedder converts the user's question into a vector so FAISS can retrieve top-k relevant chunks from your legal corpus.

6. Retrieval Helper

```
import re
import numpy as np

def _get_text(item):
    return item.get("text", "") or ""

def _get_source(item):
    return item.get("source", "") or ""

def _get_page(item):
    return item.get("page_number", "")

def retrieve_top_k_chunks(query: str, top_k: int = 5):
    q_emb = embedder.encode([query],
convert_to_numpy=True).astype(np.float32)
    D, I = index.search(q_emb, top_k)

    hits = []
    for r, idx in enumerate(I[0].tolist(), start=1):
        item = meta[idx]
        dist = float(D[0][r - 1])

        hits.append({
            "rank": r,
            "faiss_id": idx,
            "distance": dist,
            "similarity": 1.0 / (1.0 + dist),
            "text": _get_text(item),
            "source": _get_source(item),
            "page": _get_page(item),
            "chunk_id": item.get("chunk_id", idx),
        })

    return hits

def is_in_scope(question: str) -> bool:
    legal_keywords = [
        "data privacy",
        "privacy act",
        "republic act",
        "ra 10173",
```

```
        "personal information",
        "sensitive personal information",
        "data subject",
        "controller",
        "processor",
        "breach",
        "commission",
        "lawful processing",
        "disclosure",
        "penalty",
        "penalties",
        "fine",
        "imprisonment",
        "journalist",
        "secretariat",
        "portability",
        "npc"
    ]

    q = question.lower()
    return any(keyword in q for keyword in legal_keywords)

def extract_penalty_lines(context: str):
    lines = []

    for line in context.split("\n"):
        lower_line = line.lower()

        if (
            "imprisonment" in lower_line
            or "fine" in lower_line
            or "penalty" in lower_line
        ):
            cleaned = line.strip()

            if cleaned and cleaned not in lines:
                lines.append(cleaned)

    return lines
```

This function embeds the query, performs FAISS search, and returns the top-k retrieved chunks. In the demo configuration, Top-K defaults to 5 to match the Phase 2 retrieval setting.

```
import re
import numpy as np

def _get_text(item):
    return item.get("text", "") or ""

def _get_source(item):
```

```
    return item.get("source", "") or ""

def _get_page(item):
    return item.get("page_number", "")

def retrieve_top_k_chunks(query: str, top_k: int = 5):
    q_emb = embedder.encode([query],
convert_to_numpy=True).astype(np.float32)
    D, I = index.search(q_emb, top_k)

    hits = []
    for r, idx in enumerate(I[0].tolist(), start=1):
        item = meta[idx]
        dist = float(D[0][r - 1])

        hits.append({
            "rank": r,
            "faiss_id": idx,
            "distance": dist,
            "similarity": 1.0 / (1.0 + dist),
            "text": _get_text(item),
            "source": _get_source(item),
            "page": _get_page(item),
            "chunk_id": item.get("chunk_id", idx),
        })

    return hits


def is_in_scope(question: str) -> bool:
    legal_keywords = [
        "data privacy",
        "privacy act",
        "republic act",
        "ra 10173",
        "personal information",
        "sensitive personal information",
        "data subject",
        "controller",
        "processor",
        "breach",
        "commission",
        "lawful processing",
        "disclosure",
        "unauthorized disclosure",
        "malicious disclosure",
        "penalty",
        "penalties",
        "fine",
        "imprisonment",
```

```
        "journalist",
        "secretariat",
        "portability",
        "npc"
    ]

    q = question.lower()
    return any(keyword in q for keyword in legal_keywords)


def extract_penalty_lines(context: str):
    """
    Generic penalty extractor used for broad penalty/fine/imprisonment
questions.
    """
    lines = []

    for line in context.split("\n"):
        lower_line = line.lower()

        if (
            "imprisonment" in lower_line
            or "fine" in lower_line
            or "penalty" in lower_line
        ):
            cleaned = line.strip()

            if cleaned and cleaned not in lines:
                lines.append(cleaned)

    return lines


def extract_structured_penalty_clauses(context: str):
    """
    Generic extractor for structured penalty clauses.
    Designed for legal passages that enumerate penalty subclauses
    such as (a) personal information and (b) sensitive personal
information.
    """
    ctx = " ".join(context.split())
    ctx_lower = ctx.lower()

    if "unauthorized disclosure" not in ctx_lower:
        return []

    start_idx = ctx_lower.find("unauthorized disclosure")
    narrowed = ctx[start_idx:]

    next_section_match = re.search(r"section\s+\d+\.", narrowed,
```

```
flags=re.IGNORECASE)
    if next_section_match and next_section_match.start() > 0:
        narrowed = narrowed[:next_section_match.start()]

    results = []

    personal_match = re.search(
        r"a\.\s+.*?personal information.*?imprisonment ranging from.*?
fine of.*?(?=b\.|$)",
        narrowed,
        flags=re.IGNORECASE | re.DOTALL
    )
    if personal_match:
        results.append(" ".join(personal_match.group(0).split()))

    sensitive_match = re.search(
        r"b\.\s+.*?sensitive personal information.*?imprisonment
ranging from.*?fine of.*?(?=$)",
        narrowed,
        flags=re.IGNORECASE | re.DOTALL
    )
    if sensitive_match:
        results.append(" ".join(sensitive_match.group(0).split()))

    return results
```

7. Load Falcon-7B-Instruct in 4-bit

```
import os
import gc
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM,
BitsAndBytesConfig, AutoConfig
from peft import PeftModel

gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()

MODEL_ID = "tiiuae/falcon-7b-instruct"


PHASE2_ADAPTER_DIR = os.path.join(
    MODELS_DIR,
    "falcon_finetune_ckpts_v2_topk5",
    "checkpoint-10"
)
```

```
print("Adapter path:", PHASE2_ADAPTER_DIR)
print("Adapter path exists?", os.path.exists(PHASE2_ADAPTER_DIR))

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
)

print("Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID,
cache_dir=HF_CACHE_DIR)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

print("Loading Falcon base model (4-bit) with CPU offload...")
config = AutoConfig.from_pretrained(MODEL_ID, cache_dir=HF_CACHE_DIR)
config.tie_word_embeddings = False

base_model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    config=config,
    cache_dir=HF_CACHE_DIR,
    quantization_config=bnb_config,
    device_map="auto",
    low_cpu_mem_usage=True,
    torch_dtype=torch.float16,
    offload_folder="/content/offload",
    offload_state_dict=True,
)

print("Loading Phase 2 adapter...")
model = PeftModel.from_pretrained(
    base_model,
    PHASE2_ADAPTER_DIR,
    is_trainable=False,
)

model.eval()

print(" Falcon base loaded:", MODEL_ID)
print(" Phase 2 adapter loaded from:", PHASE2_ADAPTER_DIR)
print(" GPU:", torch.cuda.get_device_name(0) if
torch.cuda.is_available() else "CPU")
```

```
Adapter path: /content/drive/MyDrive/Mapua /3rd Year Term
2/CSS181-3/NLP_Project/Models/falcon_finetune_ckpts_v2_topk5/checkpoin
t-10
```

```
Adapter path exists? True
Loading tokenizer...
Loading Falcon base model (4-bit) with CPU offload...
```

{"model_id":"487f39af277e4e5b830508128d24eefd","version_major":2,"version_minor":0}

```
Loading Phase 2 adapter...
 Falcon base loaded: tiiuae/falcon-7b-instruct
 Phase 2 adapter loaded from: /content/drive/MyDrive/Mapua /3rd Year Term
2/CSS181-3/NLP_Project/Models/falcon_finetune_ckpts_v2_topk5/checkpoint-10
 GPU: Tesla T4
```

Falcon-7B-Instruct is loaded in 4-bit quantized mode with automatic device mapping and CPU/disk offloading to fit within Google Colab GPU memory constraints. After the frozen base model is loaded, the Phase 2 PEFT adapter is attached to configure the demo notebook to use the Falcon Phase 2 setup rather than the untuned baseline model.

8.Build a Strict Extraction-Style RAG Prompt

```
MAX_CHARS_PER_CHUNK = 800
MAX_TOTAL_CONTEXT_CHARS = 1800

def build_rag_prompt(question: str, retrieved_chunks: list, top_k:
int):
    blocks = []
    total = 0

    for h in retrieved_chunks:
        txt = (h["text"] or "").strip()
        txt = txt[:MAX_CHARS_PER_CHUNK]

        loc = f"{h['source']}"
        if str(h["page"]).strip():
            loc += f" (p. {h['page']})"

        block = f"[Context {h['rank']} | {loc} |
chunk={h['chunk_id']}]\n{txt}"

        if total + len(block) > MAX_TOTAL_CONTEXT_CHARS:
            break

        blocks.append(block)
        total += len(block)

    context = "\n\n---\n\n".join(blocks)
```

```
    return f"""
You are a legal QA assistant for Republic Act No. 10173
(Philippine Data Privacy Act) and related IRR documents.

STRICT RULES:
• Use ONLY the provided context. Do NOT use outside knowledge.
• If the answer is not explicitly present in the context, reply
exactly:
"Not found in the provided context."
• If the question asks to LIST conditions/items, extract them as
bullet points
  (one item per bullet).
• Do NOT repeat the same sentence or bullet point.
• When the answer contains penalties, imprisonment ranges, fines,
dates, or section numbers,
  copy them EXACTLY from the context without modification.
• Do NOT paraphrase, estimate, round, or replace numeric values.

CONTEXT (Retrieved Evidence):
{context}

QUESTION:
{question}

REQUIRED OUTPUT FORMAT:
- If listing items/conditions, output bullet points (one item per
bullet).

ANSWER:
""".strip()
```

For consistency with the Phase 2 fine-tuning configuration, the demo prompt uses the stricter fallback response “Not found in the provided context” when the answer is absent from retrieved evidence.

9. Deterministic Generation Helper

```
import torch

@torch.no_grad()
def generate_answer(prompt: str, max_new_tokens: int = 120):
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=1500,
    ).to(model.device)
```

```
    out = model.generate(
        **inputs,
        do_sample=False,
        temperature=0.0,
        top_p=1.0,
        max_new_tokens=max_new_tokens,
        use_cache=False,
        pad_token_id=tokenizer.eos_token_id,
        repetition_penalty=1.15,
        no_repeat_ngram_size=4,
    )

    input_len = inputs["input_ids"].shape[1]
    gen_text = tokenizer.decode(out[0][input_len:],
skip_special_tokens=True)
    return gen_text.strip()
```

Deterministic decoding ensures reproducibility in your demo. use_cache=False helps prevent VRAM spikes on the T4 GPU.

10. Gradio Inference Function

```
def expand_neighbors(hits, meta, window=1):
    """
    Expands retrieval results by including neighboring FAISS IDs (+/-
window).
    This helps capture adjacent legal clauses, but later filtering
will control noise.
    """
    if not hits:
        return []

    seen = set()
    expanded = []

    for h in hits:
        base_id = h["faiss_id"]

        for nid in range(base_id - window, base_id + window + 1):
            if 0 <= nid < len(meta) and nid not in seen:
                item = meta[nid]

                expanded.append({
                    "rank": h["rank"],
                    "faiss_id": nid,
                    "distance": h["distance"],
                    "similarity": h["similarity"],
```

```
                    "text": item.get("text", ""),
                    "source": item.get("source", ""),
                    "page": item.get("page_number", ""),
                    "chunk_id": item.get("chunk_id", nid)
                })
                seen.add(nid)

    return expanded


def rag_infer(question: str, top_k: int, max_tokens: int):
    question = (question or "").strip()

    if not question:
        return "Please enter a question.", ""

    if not is_in_scope(question):
        return (
            "Not found in the provided context.",
            "Question appears outside the scope of Republic Act No.
10173 and related IRR documents."
        )

    hits = retrieve_top_k_chunks(question, top_k=top_k)
    retrieved = expand_neighbors(hits, meta, window=1)

    if not retrieved:
        retrieved = hits

    if not retrieved:
        return "Not found in the provided context.", ""

    combined_context = "\n\n".join([(h["text"] or "").strip() for h in
retrieved])

    q_lower = question.lower()

    if "unauthorized disclosure" in q_lower:
        exact_lines =
extract_structured_penalty_clauses(combined_context)

        if exact_lines:
            answer = "\n".join(f"- {line}" for line in exact_lines)

            ctx_display = "\n\n".join([
                f"[{h['rank']}] sim={h['similarity']:.4f} |
{h['source']} (p. {h['page']})\n"
                f"{(h['text'] or '').strip()[:700]}..."
                for h in retrieved
            ])
```

```
            return answer, ctx_display

    if (
        "penalty" in q_lower
        or "penalties" in q_lower
        or "fine" in q_lower
        or "imprisonment" in q_lower
    ):
        penalty_lines = extract_penalty_lines(combined_context)

        if penalty_lines:
            answer = "\n".join(f"- {line}" for line in
penalty_lines[:4])

            ctx_display = "\n\n".join([
                f"[{h['rank']}] sim={h['similarity']:.4f} |
{h['source']} (p. {h['page']})\n"
                f"{(h['text'] or '').strip()[:700]}..."
                for h in retrieved
            ])

            return answer, ctx_display

    prompt = build_rag_prompt(question, retrieved,
top_k=len(retrieved))
    answer = generate_answer(prompt, max_new_tokens=max_tokens)

    ctx_display = "\n\n".join([
        f"[{h['rank']}] sim={h['similarity']:.4f} | {h['source']} (p.
{h['page']})\n"
        f"{(h['text'] or '').strip()[:700]}..."
        for h in retrieved
    ])

    return answer, ctx_display
```

This function runs the complete RAG pipeline for the live demo. It first blocks out-of-scope questions, then retrieves legal evidence, applies a direct extraction shortcut for penalty-related questions to reduce hallucinated numeric values, and otherwise falls back to standard grounded generation using the Falcon Phase 2 model.

11. Launch the Gradio UI

```
import gradio as gr

with gr.Blocks() as demo:
    gr.Markdown("# RAG Legal QA Demo (RA 10173) — Falcon Phase 2")
    gr.Markdown(
```

```
        "Ask a question about the Philippine Data Privacy Act. "
        "This demo uses the Falcon-7B Phase 2 configuration "
        "(QLoRA adapter + retrieval grounding) for live legal question
answering."
    )

    with gr.Row():
        question = gr.Textbox(
            label="Question",
            placeholder="e.g., List all conditions for lawful
processing of personal information."
        )

    with gr.Row():
        top_k = gr.Slider(
            minimum=1,
            maximum=8,
            value=3,
            step=1,
            label="Top-k retrieved chunks"
        )
        max_tokens = gr.Slider(
            minimum=64,
            maximum=300,
            value=180,
            step=16,
            label="Max new tokens"
        )

    run_btn = gr.Button("Run")

    answer_out = gr.Textbox(label="Answer (Grounded)", lines=10)
    ctx_out = gr.Textbox(label="Retrieved Context (Evidence)",
lines=14)

    run_btn.click(
        fn=rag_infer,
        inputs=[question, top_k, max_tokens],
        outputs=[answer_out, ctx_out]
    )

demo.launch(debug=False)
```

```
It looks like you are running Gradio on a hosted Jupyter notebook,
which requires `share=True`. Automatically setting `share=True` (you
can turn this off by setting `share=False` in `launch()` explicitly).

Colab notebook detected. To show errors in colab notebook, set
debug=True in launch()
```

```
Could not create share link. Please check your internet connection or
our status page: https://status.gradio.app.

<IPython.core.display.Javascript object>
```

This launches the demo UI with adjustable top-k retrieval and max generation tokens. Displaying retrieved evidence helps your professor see that answers are grounded in the legal text.